

RAG & Vector Search, Done Properly

A Complete Beginner-to-Advanced Study Guide: Chunking, Embeddings,
Hybrid Retrieval, and Reranking

July 2026

Contents

1	How to Use This Guide	3
2	Plain-English Glossary (Read This First)	3
3	Part 1: The Basics — What RAG Is and Why It Exists	4
3.1	The problem RAG solves	4
3.2	The five-stage pipeline	4
3.3	Why “naive RAG” disappoints	4
4	Part 2: Lever 1 — Chunking (Fix This First)	5
4.1	Why chunking matters more than people think	5
4.2	Chunking strategies, simple to advanced	5
4.3	Practical sizing guidance	6
5	Part 3: Lever 2 — Embeddings and Vector Search	6
5.1	How embeddings work (intuition)	6
5.2	What to know about choosing an embedding model	7
5.3	Vector databases in one page	7
5.4	The fundamental weakness of pure vector search	7
6	Part 4: Lever 3 — Hybrid Search	7
6.1	The idea	8
6.2	Merging the two result lists: Reciprocal Rank Fusion	8
6.3	Query transformation (fixing the question before searching)	8
7	Part 5: Lever 4 — Reranking (The Secret Sauce)	8
7.1	Why a second ranking pass	8
7.2	The standard production recipe	9
7.3	Assembling the final context	9
7.4	RAG vs. long context windows	9
8	Part 6: Evaluating RAG (Non-Negotiable)	10
8.1	Build a golden dataset	10

8.2 The two evaluation layers	10
8.3 The debugging matrix	10
9 Part 7: Advanced Architectures and Production Concerns	11
9.1 Agentic RAG	11
9.2 Other patterns worth knowing	11
9.3 Production checklist	11
10 Part 8: Common Mistakes, Best Practices, FAQs	12
10.1 Common mistakes	12
10.2 Best practices (condensed)	12
10.3 FAQs	12
11 Final Cheat Sheet (One-Page Revision)	13

1 How to Use This Guide

This guide teaches Retrieval-Augmented Generation (RAG) from absolute zero to production-grade. RAG is the technique behind “chat with your documents,” AI customer support that knows your help center, and search that understands meaning. Read Part 1-2 for the foundations, Part 3-6 for the four levers that decide quality, and keep the cheat sheet at the end for revision.

The one-sentence summary: *RAG means fetching the most relevant pieces of your own data and handing them to an LLM so it answers from facts instead of memory — and almost all quality problems live in the “fetching” half, not the “answering” half.*

2 Plain-English Glossary (Read This First)

- **RAG (Retrieval-Augmented Generation):** Retrieve relevant text from your data, then have an LLM generate an answer grounded in that text.
- **Corpus:** Your whole collection of documents (help articles, PDFs, wiki pages, code...).
- **Chunk:** A small piece of a document (a paragraph, a section) stored as one retrievable unit.
- **Chunking:** The strategy for cutting documents into chunks.
- **Embedding:** A list of numbers (a *vector*) that represents the meaning of a piece of text. Texts with similar meaning get vectors that are close together.
- **Vector database:** A database built to store embeddings and find “nearest” vectors extremely fast (Pinecone, Qdrant, Weaviate, pgvector, Chroma...).
- **Semantic search / dense retrieval:** Searching by meaning, using embeddings.
- **Keyword search / sparse retrieval / BM25:** Classic search that matches the actual words (BM25 is the standard ranking formula, the engine behind tools like Elasticsearch).
- **Hybrid search:** Running both semantic and keyword search and merging the results.
- **Reranker / cross-encoder:** A second, heavier model that re-scores a shortlist of retrieved chunks by reading the query and each chunk *together*, producing a much more accurate ranking.
- **Bi-encoder:** The embedding approach — query and documents are encoded *separately* into vectors. Fast but lossy.
- **Top-k:** The number of best-matching chunks you retrieve (e.g., top-5).
- **Recall:** Of all the truly relevant chunks that exist, how many did retrieval find? (Missing them = recall problem.)
- **Precision:** Of the chunks retrieved, how many were actually relevant? (Junk in the results = precision problem.)
- **Context window:** How much text an LLM can read at once, measured in tokens.
- **Grounding:** Forcing the model to base its answer on the retrieved text.
- **Hallucination:** The model inventing facts not present in the retrieved text.
- **RRF (Reciprocal Rank Fusion):** A simple formula for merging two ranked lists (e.g., keyword results + vector results) into one.
- **Metadata filtering:** Narrowing search by attributes (date, author, product, lan-

guage) before or during vector search.

3 Part 1: The Basics — What RAG Is and Why It Exists

3.1 The problem RAG solves

LLMs are trained on public data up to a cutoff date. They know nothing about *your* private documents, *your* latest product release, or anything after training. Ask a plain LLM “what is our refund policy?” and it will either admit ignorance or — worse — invent a plausible-sounding policy.

Two possible fixes:

1. **Fine-tuning:** retrain the model on your data. Expensive, slow to update, and bad at recalling exact facts.
2. **RAG:** leave the model alone; at question time, *look up* the relevant text and paste it into the prompt. Cheap, instantly updatable (just re-index the document), and every answer can cite its source.

Analogy: Fine-tuning is making an employee memorize the whole company handbook. RAG is giving the employee the handbook plus an excellent index, and teaching them to look things up before answering. The second employee is cheaper to train, always up to date, and can show you the exact page they used.

3.2 The five-stage pipeline

Every RAG system, from toy to enterprise, is this pipeline:

OFFLINE (indexing, done once per document):

Documents --> [1] Parse & clean --> [2] Chunk --> [3] Embed --> Vector DB
(+ keyword index, + metadata)

ONLINE (per user question):

Question --> [4] Retrieve (search the indexes, get top-k chunks)
--> [5] Generate (LLM answers using question + chunks)

A minimal prompt for stage 5 looks like:

You are a support assistant. Answer **ONLY** using the context below.
If the context does not contain the answer, say you don't know.

Context:

[chunk 1] ... [chunk 2] ... [chunk 3] ...

Question: {user question}

3.3 Why “naive RAG” disappoints

The 30-minute tutorial version (fixed 1,000-character chunks, plain vector search, top-3 into the prompt) demos beautifully and fails in production. Industry analyses

consistently find that when a RAG system gives a bad answer, the root cause is the **retrieval half roughly three times out of four** — the model never saw the right text, so no amount of prompt magic could save it. Naive pipelines commonly fail at retrieval on the order of 40% of queries.

This gives you the golden debugging rule of RAG:

When the answer is wrong, first look at what was retrieved. If the right chunk isn't there, you have a retrieval problem (chunking, embeddings, search, or ranking). Only if the right chunk IS there and the answer is still wrong do you have a generation problem (prompt, model).

The rest of this guide covers the four levers that fix retrieval, in order of impact: **chunking, embeddings, hybrid search, reranking** — then evaluation and advanced patterns.

4 Part 2: Lever 1 — Chunking (Fix This First)

4.1 Why chunking matters more than people think

Chunking defines the *units of knowledge* your system can ever retrieve. If a fact is split across two chunks, or buried in a chunk full of unrelated text, no embedding model or reranker downstream can fully repair the damage. Enterprises frequently treat chunking as a low-level developer detail; in practice it is one of the biggest performance levers in the whole pipeline.

Chunking controls both sides of quality:

- **Recall:** does the complete answer-bearing text live inside a single retrievable chunk?
- **Precision:** is the chunk tight enough that most of it is relevant, not padding that wastes the context window and distracts the model?

The goal in one sentence: each chunk should be *semantically complete* — able to answer a question on its own, like a well-written encyclopedia paragraph.

4.2 Chunking strategies, simple to advanced

1. **Fixed-size chunking (baseline).** Cut every N characters/tokens (e.g., 500 tokens) with some overlap (e.g., 10–15%) so sentences cut at a boundary appear whole in the next chunk. Fast to build, and its ceiling is low: it happily splits sentences mid-thought, tables mid-row, and code mid-function — producing chunks that are technically “relevant” but practically useless.
2. **Recursive / separator-aware chunking (better default).** Try to split on paragraph breaks first, then sentences, then words — only cutting mid-sentence as a last resort. Most frameworks (e.g., LangChain’s recursive splitter) do this out of the box. A big upgrade for almost no effort.
3. **Structure-aware chunking (best for structured content).** Use the document’s own structure: split Markdown/HTML on headings (one section = one

chunk), split code per function or class (using a parser/AST), keep each table intact with its header row, keep each FAQ question+answer together.

4. **Semantic chunking (best for unstructured prose).** Embed the text sentence by sentence and start a new chunk wherever the meaning shifts — detected as a drop in similarity between adjacent sentences. Practitioner comparisons report meaningful accuracy lifts over fixed-size splitting on the same data. Costs more at indexing time; often worth it for messy prose.
5. **Contextual enrichment (advanced, high payoff).** A chunk that says “It increased by 12% in Q3” is useless without knowing what “it” is. Fixes:
 - **Prepend breadcrumbs:** add the document title and section path to every chunk (“Annual Report 2025 > Revenue > Europe: It increased by 12%...”).
 - **LLM-generated context:** have an LLM write a one-sentence summary situating each chunk within its document, stored with the chunk (the idea behind “contextual retrieval”).
 - **Parent-child retrieval:** search over small, precise chunks, but hand the LLM the larger parent section they came from — small for matching, big for answering.

4.3 Practical sizing guidance

- Start around **300-800 tokens** per chunk with **10-15% overlap** for prose; tune from there with your eval set (Part 6).
- Too small → fragments without context (recall suffers). Too large → diluted meaning, embeddings average over many topics, and precision suffers.
- Always attach **metadata** to each chunk: source document, URL, section, date, author, product area, language. You will need it for filtering and citations.

Key takeaway: If you improve only one thing in an underperforming RAG system, improve chunking — bad chunks guarantee failure no matter how good everything downstream is.

5 Part 3: Lever 2 — Embeddings and Vector Search

5.1 How embeddings work (intuition)

An embedding model reads text and outputs a vector — say, 1,024 numbers — positioned so that *similar meanings land near each other*. “How do I get my money back?” and “refund policy” end up close, even though they share no words. Search then becomes geometry: embed the query, find the nearest stored vectors (by cosine similarity), return those chunks.

Analogy: imagine a giant library where books are shelved not alphabetically but by *topic proximity* — books about refunds sit next to books about returns and chargebacks. To answer a question, you walk to the point in the library matching your question’s meaning and grab the nearest books.

5.2 What to know about choosing an embedding model

- Quality varies by domain and language. Check the MTEB leaderboard, but *trust your own eval set more* — leaderboards are averages over tasks that aren't yours.
- Strong general-purpose options exist both as APIs (OpenAI, Cohere, Voyage) and open-source (the BGE and E5 families). Multilingual corpora need explicitly multilingual models.
- **Dimension trade-off:** more dimensions $\approx$ more nuance but more storage and latency. Many modern models support shortening vectors (Matryoshka embeddings) with modest quality loss.
- **Critical operational rule:** query embeddings and document embeddings must come from the *same model and version*. Changing the model means re-embedding the entire corpus.
- **Quantization** (storing vectors in lower precision, e.g., int8 or binary) can cut memory/cost dramatically with small recall loss — a standard production optimization at scale.

5.3 Vector databases in one page

A vector DB stores millions of vectors and answers “nearest neighbors?” in milliseconds using **ANN (Approximate Nearest Neighbor)** indexes — most commonly **HNSW**, a graph structure that trades a tiny bit of exactness for enormous speed. Practical picks:

- **pgvector** — if you already run PostgreSQL, start here; one less system to operate.
- **Qdrant / Weaviate / Milvus** — dedicated, self-hostable engines with filtering, hybrid support, and strong performance.
- **Pinecone** — fully managed, easiest to scale without ops work.
- **Chroma / FAISS** — great for prototypes and local development.

Choose based on your deployment model and existing infrastructure, not synthetic benchmark wars. Whatever you pick, make sure it supports **metadata filtering** (e.g., only search product = “mobile-app” and date > 2025) — pre-filtering by metadata is one of the cheapest precision wins available.

5.4 The fundamental weakness of pure vector search

A bi-encoder compresses a whole chunk into one vector — a *lossy* summary of its meaning. Consequences:

- Exact identifiers get blurred: error codes (“E-1042”), product names, acronyms, version numbers, legal citations, people’s names. Vector search may return “something about errors” instead of *that* error.
- Rare terms your embedding model barely saw in training are matched poorly.

That is exactly what keyword search is perfect at — which brings us to hybrid.

6 Part 4: Lever 3 — Hybrid Search

6.1 The idea

Run two searches in parallel and merge:

- **Dense (vector) search** catches *meaning*: “employee burnout” matches chunks about “work-life balance” and “stress management” that share no keywords.
- **Sparse (BM25 keyword) search** catches *exactness*: names, numbers, acronyms, error codes, quoted phrases.

Each covers the other’s blind spot. Hybrid search is the production standard in 2026; published comparisons show it beating semantic-only retrieval by large margins (one widely cited benchmark: ~66% vs. ~57% MRR, a 9+ point ranking-quality jump).

6.2 Merging the two result lists: Reciprocal Rank Fusion

RRF is the standard merge and is almost embarrassingly simple. Each result gets a score based only on its *rank* in each list:

$\text{RRF_score}(\text{chunk}) = \text{sum over lists of } 1 / (k + \text{rank_in_that_list})$ (k = 60 typically)

A chunk ranked #1 by keywords and #3 by vectors gets $1/61 + 1/63$. Rank-based fusion sidesteps the problem that BM25 scores and cosine similarities live on incomparable scales. Some teams instead use a weighted score blend (alpha between dense and sparse) — fine too, but it needs tuning; RRF works well untouched.

```
Query ---> BM25 keyword search ----> ranked list A \
\                                     >-- RRF merge --> top candidates
--> Embedding vector search -> ranked list B /
```

6.3 Query transformation (fixing the question before searching)

Users write bad queries. Advanced pipelines rewrite the query before retrieval:

- **Query expansion**: add synonyms/related terms, or generate several rephrasings and search with all of them.
- **HyDE (Hypothetical Document Embeddings)**: ask an LLM to write a *hypothetical answer* to the question, then search with the embedding of that fake answer — an answer-shaped query often lands closer to answer-shaped chunks than the raw question does.
- **Decomposition**: split multi-part questions (“compare our 2024 and 2025 pricing and explain the change”) into sub-queries retrieved separately.
- **Self-query / structured filters**: have an LLM extract metadata filters from the question (“in the 2025 report” → year = 2025).

7 Part 5: Lever 4 — Reranking (The Secret Sauce)

7.1 Why a second ranking pass

First-stage retrieval (hybrid) is built for speed across millions of chunks, and its ranking is only roughly right. A **reranker** takes the small candidate set — say the top 20–50 — and re-scores each one with a heavier model that reads the query and the

chunk **together**, token by token. This joint reading (a **cross-encoder**) captures fine-grained relevance that separate embeddings structurally cannot.

Analogy: first-stage retrieval is the recruiter skimming 10,000 resumes by keyword and gut feel to pick 30. The reranker is the hiring manager who actually interviews those 30. You’d never interview all 10,000 (too slow), and you’d never hire straight from the skim (too sloppy). The pipeline needs both.

7.2 The standard production recipe

The pattern the industry has converged on:

***Retrieve wide, rerank narrow, generate tight.** Retrieve the top 20-100 candidates with hybrid search (+ metadata filters) → cross-encoder reranks them → keep the top 3-10 → send those to the LLM with instructions to ground the answer and cite sources.*

Notes:

- Reranking hundreds of candidates rarely pays: the useful signal is concentrated at the head of the list, and every pair scored costs latency.
- Good reranker options (2026): **Cohere Rerank** (managed API, multilingual), **Voyage rerank** models, **BGE-reranker-v2** (open source, runs on one modest GPU), and classic MS-MARCO cross-encoders as a free battle-tested baseline.
- Rerankers add ~100-500 ms. For most Q&A products that trade is easily worth it; for strict-latency autocomplete-style features it may not be.

7.3 Assembling the final context

- Send the reranked top chunks **with their source URLs/titles** so the model can cite.
- Order can matter: models attend most to the beginning and end of the prompt (“lost in the middle”), so put the strongest chunks first and/or last.
- Instruct the model explicitly: answer only from context; say “I don’t know” when the context is insufficient; cite the source for each claim.

7.4 RAG vs. long context windows

Frontier models now offer million-token windows, tempting people to “just paste everything in.” Don’t:

- **Cost & latency:** you pay per token, every single query. Stuffing 500 pages per question is orders of magnitude more expensive than retrieving 5 relevant chunks.
- **Accuracy:** attention over huge contexts is imperfect; needle-in-haystack retrieval inside the prompt is less reliable than doing retrieval properly outside it.
- **The sensible split:** use retrieval to select; use a *generous but bounded* context (commonly ~16-64K tokens of selected material) for synthesis; reserve full-

window stuffing for tasks that genuinely require reading everything (e.g., whole-contract analysis).

8 Part 6: Evaluating RAG (Non-Negotiable)

You cannot tune what you don't measure. (The companion guide in folder 01 covers evals in depth; here is the RAG-specific core.)

8.1 Build a golden dataset

Create **50-200 representative questions**, each annotated with the chunk(s)/document(s) that contain the answer and ideally a reference answer. Source them from real user queries wherever possible. Re-run this set after *every* pipeline change, changing one variable at a time.

8.2 The two evaluation layers

Retrieval metrics (measure the search, before any LLM):

- **Recall@k** — is a correct chunk anywhere in the top k? *The single most important RAG metric*: if recall@k fails, nothing downstream can succeed.
- **Precision@k** — what fraction of the top k is relevant?
- **MRR / nDCG** — how high do correct chunks rank? (MRR rewards the first correct hit being near the top.)

Generation metrics (measure the answer, typically with an LLM judge — the RAGAS framework popularized these):

- **Faithfulness** — every claim in the answer is supported by the retrieved context (the anti-hallucination metric). Production teams commonly target > 0.9.
- **Answer relevancy** — the answer actually addresses the question (~> 0.85 as a working bar).
- **Context precision / context recall** — was the retrieved context clean and complete (~> 0.8)?

Plus the operational constraints that decide what ships: **p95 latency** and **cost per 1,000 queries**.

8.3 The debugging matrix

Symptom	Layer	Likely fix
Right doc never retrieved	Recall	Better chunking; hybrid search; query expansion; different embedding model
Retrieved but ranked too low	Ranking	Add/upgrade reranker; tune fusion weights

Symptom	Layer	Likely fix
Right chunk present, wrong answer	Generation	Stronger grounding prompt; fewer/cleaner chunks; better model
Answer ignores a table/code block	Chunking	Structure-aware splitting; keep tables intact
Fails on error codes / exact names	Sparse gap	Add BM25 to make it hybrid
Answers from outdated docs	Freshness	Metadata recency filters; index update pipeline; deduplication

9 Part 7: Advanced Architectures and Production Concerns

9.1 Agentic RAG

Instead of one fixed retrieve-then-answer pass, an LLM *agent* runs a loop: analyze the question → decide whether/what to search → read results → search again with refined queries if needed → answer when confident. This shines for **multi-hop questions** (“Which of our customers who churned in 2025 had open support tickets about billing?”) that no single query can satisfy. Cost and latency are multiples of plain RAG, so reserve it for complex queries or high-stakes domains (legal, medical, financial) — for most production use, hybrid + rerank remains the best quality-per-dollar.

9.2 Other patterns worth knowing

- **GraphRAG**: build a knowledge graph of entities/relations from the corpus; retrieval walks the graph. Strong for “how is X related to Y” and whole-corpus summary questions.
- **Multi-vector / late-interaction (ColBERT, ColPali)**: represent each chunk with many small vectors instead of one, recovering token-level matching precision; ColPali extends this to *document images*, retrieving over pages with charts and tables directly.
- **Routing**: classify the query first and send it to the right index/pipeline (product docs vs. HR policies vs. SQL over the database).

9.3 Production checklist

- **Index freshness**: an update pipeline (webhooks/CDC or scheduled re-crawls) so edited docs are re-chunked and re-embedded; delete removed docs (stale chunks are a top source of confidently wrong answers).
- **Access control**: filter chunks by the *asking user’s* permissions at query time — never let embeddings leak documents the user can’t open.
- **Observability**: trace every stage (query, retrieved chunks + scores, reranked list, final prompt, answer) so you can debug any bad answer after the fact.

- **Security:** treat retrieved content as untrusted input — a document can contain prompt-injection text (“ignore previous instructions...”); constrain the model’s tools and instructions accordingly.
- **Caching:** cache embeddings of repeated queries and even full answers for hot questions.

10 Part 8: Common Mistakes, Best Practices, FAQs

10.1 Common mistakes

1. **Tuning the prompt when retrieval is broken** — always inspect retrieved chunks first; retrieval causes most failures.
2. **Fixed-size chunking forever** — the fast start with the low ceiling; splitting tables, code, and sentences mid-thought.
3. **Vector-only search** — silently terrible on names, codes, and acronyms; hybrid is the standard for a reason.
4. **No reranker** — leaving the single biggest post-retrieval quality win on the table.
5. **No golden dataset** — every change is a guess; regressions ship invisibly.
6. **Stuffing top-20 raw chunks into the prompt** — noise in, noise out; rerank down to a clean 3-10.
7. **Ignoring metadata** — dates, products, and permissions are precision and safety gold.
8. **One-time indexing** — the corpus changes; a RAG system is a living system, not a one-off build.
9. **Skipping the “say I don’t know” instruction** — the cheapest hallucination reducer there is.

10.2 Best practices (condensed)

- Chunk by structure; enrich chunks with titles/breadcrumbs; ~300-800 tokens, 10-15% overlap as a starting point.
- Hybrid (BM25 + vectors) fused with RRF; metadata filters wherever applicable.
- Retrieve ~20-100 → rerank → send top 3-10 with sources; strongest chunks at the edges of the prompt.
- Ground hard: answer only from context, cite sources, admit ignorance.
- Golden dataset of 50-200 queries; track recall@k, faithfulness, relevancy, p95 latency, cost; change one variable at a time.
- Trace everything; secure everything (permissions + injection awareness); keep the index fresh.

10.3 FAQs

Q: RAG or fine-tuning? A: For *knowledge* (facts that change, need citations), RAG. For *behavior* (style, format, domain jargon), fine-tuning. Many systems use both: a fine-tuned model inside a RAG pipeline.

Q: What top-k should I use? A: Retrieve wide (20-100), rerank, and pass 3-10 to the model. Tune with recall@k on your golden set — the answer is empirical, not

universal.

Q: Do I need a dedicated vector database? A: Not at first. pgvector on your existing Postgres handles millions of vectors fine. Move to a dedicated engine when scale, latency, or hybrid/filtering features demand it.

Q: How do I handle PDFs with tables and charts? A: Parsing quality is a real battle: use a strong document parser, keep tables intact as chunks (or convert to Markdown), and for chart-heavy corpora consider vision-based retrieval (ColPali-style) or having a vision model describe figures at indexing time.

Q: Why does my bot answer from an old version of a document? A: Both versions are in the index. Add version/date metadata, filter or deduplicate at query time, and delete superseded chunks during re-indexing.

Q: Is RAG dead now that context windows are huge? A: No — cost, latency, and in-context attention limits all argue for selecting before reading. Long context changed *how much* you can pass to the model, not *whether* you should select it well.

11 Final Cheat Sheet (One-Page Revision)

The pipeline: Parse → Chunk → Embed (+ BM25 index, + metadata) → Retrieve (hybrid) → Rerank → Generate (grounded, cited).

The golden debugging rule: wrong answer → look at retrieved chunks first. ~3 of 4 failures are retrieval, not generation.

The four levers, in order of impact: 1. **Chunking** — structure-aware, semantically complete units; ~300–800 tokens; add breadcrumbs/metadata; parent-child for context. 2. **Embeddings** — pick per your domain/language via your own evals; same model for queries & docs; quantize at scale. 3. **Hybrid search** — dense (meaning) + BM25 (exact terms), merged with RRF (score = sum of $1/(60+\text{rank})$); metadata pre-filters. 4. **Reranking** — cross-encoder over top 20–100, keep 3–10. “Retrieve wide, rerank narrow, generate tight.”

Grounding prompt essentials: answer only from context • cite sources • say “I don’t know” when context is insufficient • strongest chunks first/last.

Metrics & targets (working bars): recall@k (the king) • faithfulness > 0.9 • answer relevancy > 0.85 • context precision/recall > 0.8 • p95 latency • cost/1k queries. Golden set: 50–200 real queries; one change at a time.

Advanced toolbox: query expansion • HyDE • decomposition • self-query filters • parent-child • contextual chunk summaries • agentic RAG (multi-hop) • GraphRAG (relations) • ColBERT/ColPali (late interaction, visual docs).

Production musts: freshness pipeline • permission filtering • full tracing • injection awareness • caching.

The one rule: *An LLM can only be as truthful as the chunks you hand it — spend your effort where the failures are: retrieval.*